Local AI with Ollama

Practical Lab Report — Running and Developing Local AI Applications

Student: Yassir · **Level:** PhD · **Module:** Artificial Intelligence / Generative AI · October 2026

Code and notebook: github.com/yassirtl/Local-AI-Ollama-Lab

1. Environment and setup

I did the lab on Google Colab with a T4 GPU. On Colab, "local" means local to the Colab VM: the model runs on the T4 of the machine and the notebook talks to it through `localhost:11434`, exactly like on a laptop.

Item	Value
Machine	Google Colab VM, Linux 6.6, Python 3.13
GPU	NVIDIA Tesla T4, 15 GB (15360 MiB)
Ollama	0.40 (Linux install script)
Model 1	`qwen3.5:9b` (9.0B parameters, Q4_K_M, 6.6 GB)
Model 2	`gemma3:4b` (4B parameters, 3.4 GB, also used for vision)
Embeddings	`nomic-embed-text` (274 MB)

Two things were different from a normal installation. The install script needs `zstd` to unpack Ollama, so I installed it first. And Colab has no systemd and no second terminal, so the server does not start by itself: I started `ollama serve` as a background process from Python and waited until `localhost:11434` answered.

```
!ollama --version
print("Ollama running:", ollama_running())
!nvidia-smi --query-gpu=name,memory.total --format=csv
```

```
ollama version is 0.40.11
Ollama running: True
name, memory.total [MiB]
Tesla T4, 15360 MiB
```

Figure 1 — Ollama installed and running, on a Tesla T4

```
print(ask("Say hello in one short sentence.", think=False))
!ollama list
!ollama ps
```

```
[29.5s]
Hello!
NAME                        ID              SIZE      MODIFIED
nomic-embed-text:latest     0a109f422b47    274 MB    28 minutes ago
gemma3:4b                   68ceab7e468a    3.4 GB    37 minutes ago
qwen3.5:9b                  56671c2ab938    6.6 GB    47 minutes ago
NAME          ID              SIZE      PROCESSOR    CONTEXT    RUNNER      UNTIL
qwen3.5:9b    56671c2ab938    5.6 GB    100% GPU     4096       llamacpp    4 minutes from now
```

Figure 2 — Downloaded models (`ollama list`) and qwen3.5:9b loaded at 100% GPU (`ollama ps`)

2. First local LLM (Part 3)

1. Did the model produce different answers? Why? Yes, very different. The three prompts ask for the same topic but for a different audience, and the model adapts the vocabulary, the length and the structure:

- "five sentences": exactly five plain sentences (definition, learning from data, applications, future).
- "10-year-old child": a "smart robot friend" analogy, examples from daily life (Netflix, YouTube, Siri/Alexa), and a reassuring tone ("AI is not perfect (and that's okay!)").
- "computer science professor": formal vocabulary (symbolic AI vs connectionism, distributional shift, inductive bias, causal reasoning) and a structured answer with sections.

There is also a second reason: even with the same prompt, the text would change at each run, because the model *samples* the next token. `ollama show` (Part 5) gives the sampling parameters of qwen3.5: `temperature 1`, `top_k 20`, `top_p 0.95`, so the output is not deterministic.

2. Is the model communicating with a cloud API? No. `generate()` sends a POST request to `http://localhost:11434/api/generate`, which is the Ollama server running on the same machine. There is no API key and no external host in the code. The Internet was used only once, to download the model weights with `ollama pull`.

3. Where is inference being performed? On the T4 GPU of the Colab machine. `ollama ps` shows `qwen3.5:9b`, `5.6 GB`, `100% GPU`, with a context of 4096 tokens: all the weights are in GPU memory and the llama.cpp runner of Ollama computes the tokens there.

About the time: the first answer took 150 s, the next ones 40 s and 75 s. The first call includes loading the model into the GPU, and qwen3.5 is a "thinking" model (thinking is on by default), so it writes a hidden reasoning before the answer. This reasoning is not in `response` but it costs time.

4. What happens if the Internet connection is disabled? On a laptop, the cells above keep working: the model is on the disk and the request never leaves the computer. Only `ollama pull` (and installing Ollama) needs the Internet. On Colab I could not test it, because the browser talks to the Colab VM through the Internet, so cutting the connection also cuts the notebook. But the code shows the same thing: the only address used is `localhost`, so the model does not depend on any online service once it is downloaded.

3. Prompt engineering (Part 4)

Both prompts use the same role ("You are a university professor"), but the answers are very different.

- **Prompt A** (120 s) gave a formal lesson that follows the 5 requested points exactly, as 5 headings: definition with sets V and E, an example (course prerequisites), an adjacency matrix in LaTeX, build systems and Git as applications, and a Python adjacency list.
- **Prompt B** (108 s) gave an informal explanation ("Pull up a chair", "No time machines!") with one example (Math 101 → CS 101 → Algorithms 201), no matrix and no code.

Role of each element:

- **Role:** sets the voice. In both answers the model starts like a teacher in front of a class ("Hello class! Please take a seat.").

- **Context:** who the answer is for. "First-year student" in B changed the level: simple words, everyday analogies, no formal notation.
- **Constraints:** what must be in the answer. The list in A forced the matrix and the Python example; B had none of them because they were not asked.
- **Output format:** the numbered list in A became the structure of the answer (sections 1 to 5). In B the model chose its own structure and added parts I did not ask for ("Why should you care?").
- **Specificity:** the more precise the prompt, the more predictable and complete the answer. B was more open, so the model decided more by itself.

A precise prompt does not make the content correct, though. In A the model writes that in Git "every commit has one parent", which is wrong: a merge commit has two parents (and that is exactly why Git history is a DAG and not a tree).

4. Model management (Part 5)

- `ollama list` shows the models **stored on the disk**: name, ID, size (6.6 GB for qwen3.5:9b) and date.
- `ollama show` describes **one model**: architecture `qwen35`, 9.0B parameters, quantization `Q4_K_M`, maximum context 262144 tokens, its capabilities (completion, vision, tools, thinking), the vision projector (CLIP, 456M parameters), the default sampling parameters and the license.
- `ollama ps` shows the models **currently in memory**: size in memory (5.6 GB), where they run (`100% GPU`), the context allocated (4096) and when they will be unloaded ("4 minutes from now").

Downloaded vs loaded: a downloaded model is just a file on the disk. It uses disk space but nothing else. A loaded model is copied into the GPU (or RAM) and ready to answer at once. Ollama unloads a model after about 5 minutes without requests (`keep_alive`), and the next request then has to load it again. In the benchmark (Part 16) I forced this "cold start": loading took 35.8 s for qwen3.5:9b and 13.0 s for gemma3:4b, before the first token.

The size in memory (5.61 GB) is not the same as the file size (6.55 GB). What is in memory depends on what Ollama actually allocates (for example the context of 4096 tokens), so it is not simply the size of the file.

5. Comparing two local models (Part 6)

Scores (after reading the answers and testing the code):

Model	Time (s)	Quality /5	Code correctness /5
qwen3.5:9b	40.3	5	5
gemma3:4b	43.3	4	5

- **Which model is faster?** Here the times are almost the same (40.3 s vs 43.3 s), because gemma3 wrote a longer answer (example calls and a longer explanation). Per token, gemma3 is about **2 times faster** (51.6 vs 26.2 tokens/s in Part 16). The total time depends on the speed *and* on the length of the answer.
- **Which produces better code?** Both functions are correct: I pasted them in a test cell in the notebook and they give the same result as the reference for every number from -5 to 19 999. qwen3.5's version is a bit better: it handles 2 and even numbers first and then tests only odd

divisors (about √n/2 checks instead of √n), and its docstring has doctest examples. gemma3 tests every number from 2 and puts `print()` calls outside the function, which would run each time the file is imported. Both use `n ** 0.5`, a float; `math.isqrt(n)` would be safer for very large numbers.

- **Which uses more memory?** qwen3.5:9b: 5.6 GB of VRAM, against 2.9 GB for gemma3:4b (Part 16).
- **Which would I choose for programming assistance?** On this T4, qwen3.5:9b: cleaner code, and it can use thinking mode for harder problems. On a laptop with little memory, gemma3:4b is good enough for simple functions like this one.
- **Is the largest model necessarily the best?** No. On this task both models gave correct code; the bigger one needs twice the memory and is half as fast. A bigger model helps on harder tasks, but you have to test it on your own task and hardware.

6. Python client, chatbot and memory (Parts 7 to 9)

The script `local_ai_lab/app.py` sends one POST request to `/api/generate` and prints the answer. It is a separate Python process, so it shows that any program can use the model through the REST API.

```
1 !python local_ai_lab/app.py

### 5. Important Limitations & Risks

#### A. Infinite Recursion
If your **Base Case** is missing or impossible to reach (e.g., incrementing a counter inside the function instead of decrementing), Python will cr

**Crash Example:**
```python
def infinite_loop(n):
    return n + 1 # Recursive step, but never calls itself
    # This crashes immediately, doesn't crash infinitely
    # if you called it with an argument without a base case inside the logic
```

Bad Recursion Example (Infinite):
```python
# Missing Base Case!
def bad_factorial(n):
    return n * bad_factorial(n - 1)
# Python calls itself forever until the system crashes.
```

B. Maximum Recursion Depth
Python limits recursion depth to prevent the stack from overflowing the memory. By default, this is **999 frames**.
If your data requires more depth (e.g., a linked list that is 1000+ items long), you will get a `RecursionError`.

Check Limit:
```python
import sys
print(sys.getrecursionlimit()) # Default usually 1000
```
```

Figure 3 — `python local_ai_lab/app.py`: a separate script gets an answer from the local model (end of the long answer)

Then I built the chatbot, first without memory and then with a `history` list.

```
1 #03: chatbot (a text box appears under the cell)
2 history.clear()
3 chat_loop_memory()
4

You:  My name is Yassir

AI: Nice to meet you, Yassir! How can I assist you today?

You: What is my name?

AI: Your name is Yassir! I remember because you told me at the beginning of our conversation.

You:
```

Figure 4 — Interactive chatbot with conversation history: the model answers with the name given earlier

Why does the model remember the name? The LLM does not remember anything. The weights of the model do not change between two calls, and each call to `/api/generate` is independent.

The "memory" is the Python list `history`. At each question we rebuild the prompt with all the previous messages, as the printed prompt shows:

```
User: My name is Ahmed.
Assistant: Hello Ahmed! It's nice to meet you. ...
User: What is my name?
Assistant:
```

So when the model answers "Your name is Ahmed!", it is just reading the name in its own input.

Consequences: the prompt gets longer at every turn, so the answers get slower, and it is limited by the context window (4096 tokens here, see `ollama ps`). After that, the oldest messages are lost. `/api/chat` works the same way: we send the whole list of `messages` each time; the only difference is that the roles are clear and Ollama applies the chat template of the model for us.

7. Code assistant (Part 10)

For the `Calculator.divide` method, qwen3.5:9b found the main problem (`ArithmeticException` when `b == 0`), explained integer division truncation, and proposed a corrected class with a check on `b`, Javadoc and a `double` version. Its corrected code imports `AtomicInteger` without using it, and it did not mention the other overflow case (`Integer.MIN_VALUE / -1`).

For the `Student` class it found the bug right away: `age = age;` assigns the parameter to itself because the parameter hides the field, so the field stays 0. The fix is `this.age = age;`. It also added a missing `getAge()` and input validation.

8. Structured output (Part 11)

What I observed:

- With the prompt only ("Return ONLY JSON"), the model returned the JSON inside a ```` ```json ```` block, so `json.loads()` on the raw text would fail; I needed `extract_json()` with a regex. Also `risk` was a long sentence and not a category.
- With the JSON Schema in `format`, the output was valid JSON directly, with the right types (`age` is an `int`, `grade` a `float`) and `risk` limited to `low` / `medium` / `high`.

Why it matters: a program does not read text like a human. It needs a fixed contract: which fields, which types, which values. With structured output we can parse the answer without tricks, check it, store it in a database, or trigger an action (for example if `risk == "high"`). Without it, one extra sentence or a code fence from the model breaks the application.

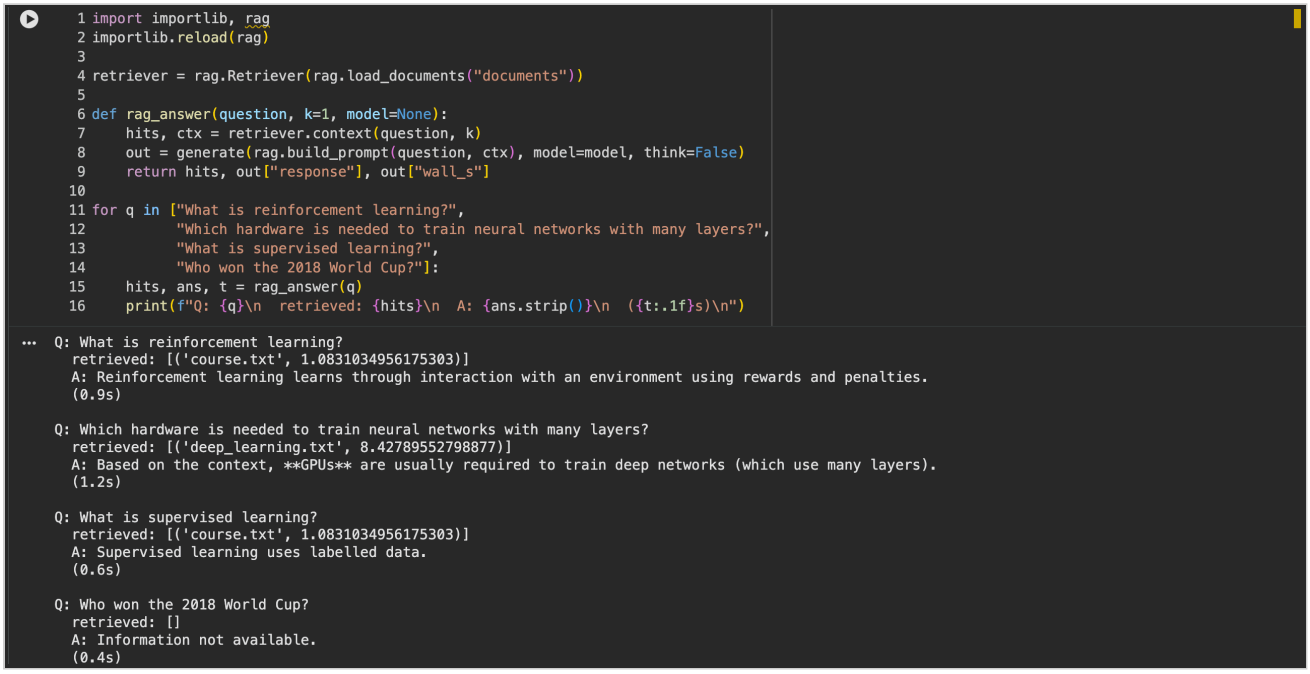
But the schema controls the **form**, not the **truth**. In my run, the constrained answer said `"risk": "high"` and "grade significantly below age expectancy" for a student with 15.5 and 2 absences, while the first answer said "Low risk". The prompt never says the grade is out of 20, so the model guessed. We still need to validate the content and give enough context in the prompt.

9. Document assistant and mini-RAG (Parts 12 and 13)

Yes, here it worked both times. For "What is reinforcement learning?" the model answered with the sentence of the document, almost word for word. For "What is quantum computing?" it answered exactly "Information not available.", even though it surely knows what quantum computing is from its training. It was also fast (0.9 s and 0.3 s) because the answers are short and thinking is off.

The same happened in the mini-RAG (Part 13): "Who won the 2018 World Cup?" → "Information not available."

It is still only an instruction, not a guarantee. With a long document, a question close to the document, or a smaller model, the model can mix in what it knows from its training. That is why real systems also show the sources, so the user can check.



```
import importlib, rag
importlib.reload(rag)

retriever = rag.Retriever(rag.load_documents("documents"))

def rag_answer(question, k=1, model=None):
    hits, ctx = retriever.context(question, k)
    out = generate(rag.build_prompt(question, ctx), model=model, think=False)
    return hits, out["response"], out["wall_s"]

for q in ["What is reinforcement learning?",
          "Which hardware is needed to train neural networks with many layers?",
          "What is supervised learning?",
          "Who won the 2018 World Cup?"]:
    hits, ans, t = rag_answer(q)
    print(f"Q: {q}\n  retrieved: {hits}\n  A: {ans.strip()}\n  ({t:.1f}s)\n")
```

```
Q: What is reinforcement learning?
  retrieved: [('course.txt', 1.0831034956175303)]
  A: Reinforcement learning learns through interaction with an environment using rewards and penalties.
  (0.9s)

Q: Which hardware is needed to train neural networks with many layers?
  retrieved: [('deep_learning.txt', 8.42789552798877)]
  A: Based on the context, **GPUs** are usually required to train deep networks (which use many layers).
  (1.2s)

Q: What is supervised learning?
  retrieved: [('course.txt', 1.0831034956175303)]
  A: Supervised learning uses labelled data.
  (0.6s)

Q: Who won the 2018 World Cup?
  retrieved: []
  A: Information not available.
  (0.4s)
```

Figure 5 — Mini-RAG: retrieved document, answer and time for each question; no document for the World Cup question

The keyword retriever has limits. For "What is reinforcement learning?" it picks `course.txt` and not `reinforcement_learning.txt`, and for "What hardware accelerates network training?" two documents get exactly the same score (0.875). With embeddings (`nomic-embed-text`), `deep_learning.txt` comes first with a clear margin (0.64 vs 0.53).

10. Vision (Part 14)

- **LLM vs multimodal model:** a classic LLM takes text and produces text. A multimodal model also accepts images: a vision encoder turns the image into vectors that are given to the language model together with the text tokens. `ollama show` (Part 5) shows this part for qwen3.5: a *Projector* of type `clip` with 456M parameters, which maps the image features into the 4096-dimension space of the LLM.
- **What extra information it receives:** the image itself, sent in base64 in the `images` field of the request, which the model turns into "visual tokens". With it, gemma3:4b (24.2 s) found the tabby cat, the stone wall, the bare branches, the blue sky, an outdoor scene in daylight, and correctly said

there is no text in the image. The "interpretation" part is more of a guess (winter or late fall because the branches are bare, the cat looks "contemplative").

- **Applications of local vision AI:** analysis and OCR of private documents (invoices, exam copies, ID cards) without uploading them; quality inspection in a factory, where machines are often offline; education (explaining a diagram or a photo); preprocessing of medical images when data must stay in the hospital; video surveillance analysis, which raises privacy and ethical questions. In most of these cases the images are sensitive, so running locally is a real advantage.

11. Web application and final project (Parts 15 and 17)

The final project, the *FSTT Local AI Assistant*, is in `LocalAI_Lab/source/`: a Flask app, the `rag.py` retriever, the console `chatbot.py`, `benchmark.py` and the HTML template.

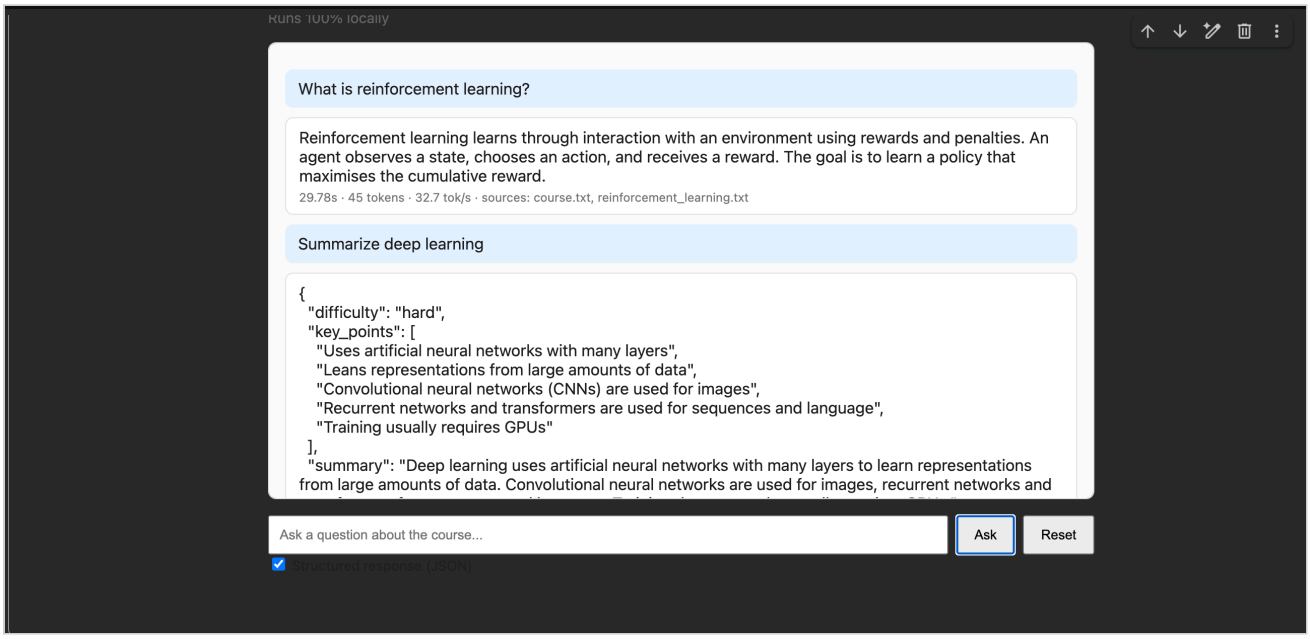


Figure 6 — FSTT Local AI Assistant: an answer from the course documents with its sources and time, and a structured (JSON) answer

On Colab the Flask app had to listen on `0.0.0.0`, because the Colab proxy that shows the page does not reach an app listening only on `127.0.0.1`. The default in the code stays `127.0.0.1`, and the address is chosen with the `HOST` variable.

12. Performance benchmark (Part 16)

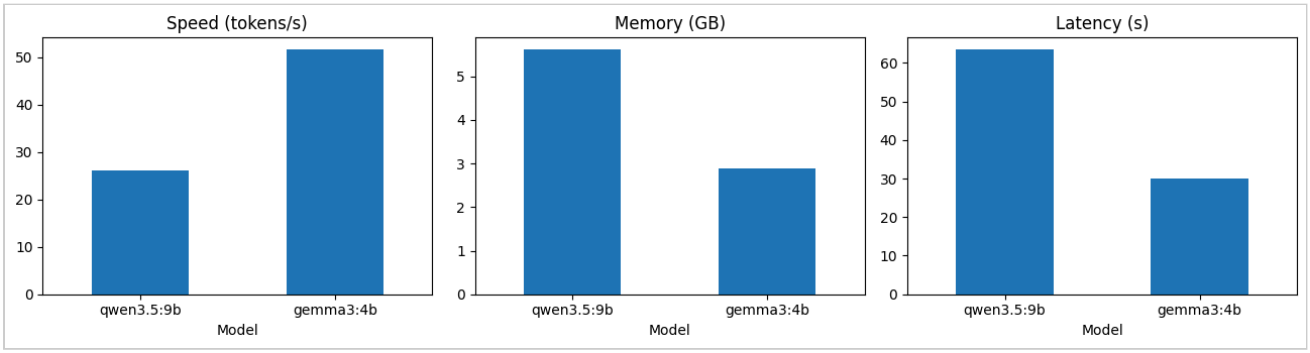


Figure 7 — Speed, memory and latency of the two models on the T4

Measured on the Colab T4 (cold start, same 500-word prompt, thinking off):

Model	Disk (GB)	In VRAM (GB)	Load (s)	Total time (s)	Tokens	Tokens/s
qwen3.5:9b	6.55	5.61	35.8	63.4	713	26.2
gemma3:4b	3.35	2.88	13.0	29.9	856	51.6

- **Speed:** gemma3:4b generates about 2 times more tokens per second, and it is the model that is half the size. That is not a coincidence: to generate one token the GPU has to read all the weights from its memory, so speed is limited by memory bandwidth. 26.2 tok/s × 5.61 GB ≈ 147 GB/s and 51.6 tok/s × 2.88 GB ≈ 148 GB/s: both models read the weights at the same rate, about half of the 320 GB/s of a T4.
- **Memory:** 5.6 GB vs 2.9 GB of VRAM. Both fit in the 15 GB of the T4 at the same time, but on an 8 GB laptop the 9B model would leave little room.
- **Latency:** the total time includes loading the model (35.8 s for qwen3.5). Without the loading, the generation itself takes about 27 s for qwen3.5 and 17 s for gemma3. In normal use the model stays loaded, so only the first request pays this cost.
- **Quality:** `bench()` only keeps the counters, not the text, so the quality score comes from the other parts (Part 6 code, Part 10, Part 14): 5 for qwen3.5, 4 for gemma3.
- **RAM:** the "System RAM used" column is for the whole machine, measured after each run, so it does not isolate one model. The VRAM column is the reliable one.

Conclusion: the bigger model gives better answers but costs 2× memory and 2× time per token. For a T4, qwen3.5:9b is a good choice; for a small machine, gemma3:4b gives most of the quality at half the cost.

13. Answers to the report questions

Q1. What is Local AI?

Local AI means running the AI model on your own machine (laptop, server, or here the Colab VM) instead of sending the requests to a provider's cloud. The model weights are downloaded once, and then the prompts, the documents and the answers stay on the machine. In this lab: Ollama running `qwen3.5:9b` and `gemma3:4b` on a T4 GPU, used from Python through `localhost:11434`.

Q2. Difference between a Cloud LLM and a Local LLM?

	Cloud LLM	Local LLM
Where it runs	Provider's servers	My machine
Data	Sent over the Internet to a third party	Stays on the machine
Cost	Pay per token / subscription	Hardware + electricity, no cost per token
Internet	Required	Only to download the model
Model size	Very large (hundreds of billions of parameters)	Limited by my memory (here 4B and 9B)
Quality	Usually better	Lower, depends on the model
Speed	Fast, big GPUs	Depends on my hardware (26 to 52 tok/s on a T4)
Control	Provider can change or remove the model	I choose and keep the exact version

Q3. What role does Ollama play?

Ollama is the program between my application and the model. It:

- downloads models from its library and stores them (`ollama pull`, `ollama list`);
- loads them into the GPU or the RAM, decides how much goes on the GPU, and unloads them after some idle time (`ollama ps`, `keep_alive`);
- runs the inference with its engine (llama.cpp runner here) and applies the chat template of each model;
- exposes everything as a REST API on port 11434 (`/api/generate`, `/api/chat`, `/api/embed`, `/api/tags`, `/api/ps`, `/api/show`), with options like structured outputs and thinking on/off.

It is a bit like Docker for models: one command to get a model and one standard interface to use any of them.

Q4. What is model quantization?

Quantization means storing the weights of the model with fewer bits. A model is normally trained in 16 or 32-bit floats. With `Q4_K_M` (our qwen3.5:9b), most weights use about 4 bits: they are grouped in blocks, and each block keeps a scale factor to rebuild approximate values. The "M" (medium) variant keeps some sensitive tensors at higher precision.

Example: 9 billion parameters in 16 bits would take about 18 GB, more than the T4 has. Quantized, the file is 6.6 GB and it runs at 26 tokens/s. We lose a little quality but gain a lot of memory and speed.

Q5. Why does model size affect memory consumption?

Because all the weights have to be in memory to generate each token. Memory needed ≈ number of parameters × bytes per parameter, plus the KV cache (the memory of the context, which grows with the context length) and some buffers. That is why the 9B model takes 5.6 GB of VRAM and the 4B model 2.9 GB, both in 4-bit quantization. If the model does not fit in the GPU, Ollama puts part of it on the CPU and it becomes much slower.

Q6. Why can a smaller model sometimes be preferable?

- It fits on machines with little memory (gemma3:4b uses 2.9 GB, qwen3.5:9b 5.6 GB).

- It is faster: 51.6 vs 26.2 tokens/s, and 13 s vs 36 s to load in my benchmark.
- For simple tasks it is good enough: gemma3:4b wrote a correct `is_prime`, found the right objects in the image, and for RAG the answer comes from the documents anyway, so the model does not need a lot of knowledge.
- It uses less energy and leaves room for other programs or other models on the same GPU.

Q7. Advantages of local inference?

- **Privacy:** the data never leaves the machine (important for medical, legal or student data, and for GDPR).
- **Offline:** it works without Internet once the model is downloaded.
- **Cost:** no price per token, so you can run as many requests as you want (the benchmark, the tests...).
- **Control:** I pin an exact version (`qwen3.5:9b`), nobody can change or remove it, and there are no rate limits.
- **Customization:** system prompts, parameters, choice of model for each task.

Q8. Disadvantages of local inference?

- **Hardware:** you need enough RAM / VRAM; big models are impossible on a normal laptop.
- **Quality:** small models make more mistakes. In this lab: "every Git commit has one parent" (Part 4), or a student with 15.5 marked "high risk" (Part 11).
- **Speed:** on weak hardware it is slow. Even on the T4, the first answer with thinking took 150 s.
- **Work for me:** installation, updates, monitoring, security; there is no team behind it like with a cloud API.
- **Scaling:** one GPU serves a few users, not thousands.
- **Energy and disk:** the models take several GB each.

Q9. Explain the architecture of your Python application.

The final app (Part 17) has these layers:

1. **Browser:** `index.html` with a little JavaScript that sends the question to `/ask` with `fetch()` and shows the answer, the sources, the time and the tokens/s.
2. **Flask (`app.py`):** three routes: `/` (the page), `/ask` (one question) and `/reset` (clear the history).
3. **Retriever (`rag.py`):** a keyword TF-IDF score over the files of `documents/`; the 2 best documents become the CONTEXT.
4. **Prompt building:** a system message (answer only from the context, otherwise "Information not available."), the last 6 exchanges from `history`, and the new question with its context.
5. **Ollama:** POST to `/api/chat` with `think: false`, and a JSON Schema in `format` when the "structured" box is checked.
6. **Response:** Flask stores the question and the answer in the history (the question without the context, to keep it small) and returns JSON with the answer, sources, seconds and tokens/s.

In the test, the app answered in 0.7 to 7.5 s, understood the follow-up question "And how is it different from supervised learning?" thanks to the history, returned a valid JSON summary of deep learning, and answered "Information not available." for quantum computing.

Q10. Explain the role of the REST API.

The REST API is the interface between my code and the model. My program only sends an HTTP request with JSON (model, prompt or messages, options) and receives JSON (the answer and counters like `eval_count`). So:

- any language or tool can use the model: Python `requests`, Flask, JavaScript, `curl`;
- the application does not depend on how the model runs: to change the model I change one string, and to use another machine I change `OLLAMA_URL`;
- the same API is used for everything: generation, chat, embeddings, list of models, loaded models.

The API is stateless: each request must contain everything the model needs, which is why we resend the history (Part 9).

Q11. What is RAG?

RAG (Retrieval-Augmented Generation) means: first **retrieve** the documents related to the question, then put them in the prompt, and let the model **generate** the answer from them. It lets the model answer about information it never saw in training (a course, private files), reduces hallucinations, allows showing the sources, and the knowledge can be updated just by changing the documents, without training.

In our mini-RAG the retriever is a simple keyword TF-IDF. It worked (GPUs for deep learning, refusal for the World Cup), but I saw its limits: for "What is reinforcement learning?" it picked `course.txt` instead of `reinforcement_learning.txt`, and for "What hardware accelerates network training?" two documents got exactly the same score (0.875). With embeddings (`nomic-embed-text`) the semantic search clearly put `deep_learning.txt` first (0.64), because it compares meanings and not words. Real RAG systems use embeddings, cut documents into chunks and store them in a vector database.

Q12. Difference between an LLM and a multimodal LLM?

An LLM only works with text (text in, text out). A multimodal LLM also takes other inputs, here images: a vision encoder (for example CLIP) turns the image into vectors and a projector maps them into the embedding space of the LLM, so the model reads the image as extra tokens next to the text. In Part 14, gemma3:4b received the photo in base64 and described the cat, the wall and the sky. qwen3.5:9b is multimodal too: `ollama show` lists the `vision` capability and a CLIP projector of 456M parameters.

Q13. Compare two local models experimentally.

Same T4 GPU, same prompts:

	qwen3.5:9b	**gemma3:4b**
Parameters / quantization	9.0B / Q4_K_M	4B / Q4_K_M
Disk / VRAM	6.55 GB / 5.61 GB	3.35 GB / 2.88 GB
Load time (cold)	35.8 s	13.0 s
Speed	26.2 tokens/s	51.6 tokens/s
`is_prime` (Part 6)	correct, 40.3 s	correct, 43.3 s
Quality /5	5	4

gemma3:4b is twice as fast and uses half the memory; qwen3.5:9b writes cleaner and more complete answers and has a thinking mode. Speed × size is almost constant (≈ 147 GB/s for both), which shows

that generation is limited by the memory bandwidth of the GPU.

Q14. Which model would you deploy on a computer with limited RAM? Explain.

gemma3:4b. Loaded, it needs 2.9 GB, against 5.6 GB for qwen3.5:9b, and on a computer with limited RAM you must also keep memory for the system, the context and the other programs. With 8 GB of RAM and no GPU, the 9B model would fill almost everything and become very slow. In my tests gemma3:4b was still correct on the code and on the image, and it is twice as fast. With even less memory (4 GB), I would take `gemma3:1b` or `qwen3.5:2b`, and accept lower quality.

Q15. What security risks exist when exposing a local LLM server to a network?

By default Ollama only listens on `127.0.0.1` (the install script says it: "The Ollama API is now available at 127.0.0.1:11434"), so only the same machine can use it. If you set `OLLAMA_HOST=0.0.0.0`, anyone who can reach port 11434 can use it, because **there is no authentication**:

- **Use of my compute:** anyone can send requests and use my GPU, or flood it so it is unavailable for me.
- **Model management:** the API can also pull, copy and delete models (`/api/pull`, `/api/delete`) and list what I have.
- **Data exposure:** the prompts often contain private data. In our Flask app the `history` is one global list, so a second user would get the conversation of the first one in their context.
- **Prompt injection through documents:** a file in `documents/` containing "ignore the previous instructions..." goes straight into the prompt.
- **No TLS:** traffic is plain HTTP, so it can be read on the network.
- **Flask debug mode:** the Part 15 app runs with `debug=True`. The Werkzeug debugger is an interactive Python console in the browser, protected only by a PIN; it must never be reachable from a network.

Protection: keep Ollama on `localhost`; if remote access is needed, put a reverse proxy (nginx, Caddy) in front of it with authentication and HTTPS, or use a VPN / SSH tunnel; block port 11434 with a firewall; turn off debug mode; one history per user; and treat the documents as untrusted input.

On Colab I had to start the Flask app with `HOST=0.0.0.0`, because the Colab proxy does not reach an app listening only on `127.0.0.1`. That was acceptable there because the VM has no public port: the page is only reachable through the Colab proxy, after logging in to my Google account. On a laptop or a server on a real network, I would keep `127.0.0.1`.

14. Conclusion

This lab showed that a useful assistant can run entirely on one machine: Ollama downloads and serves the model, Python talks to it through a simple REST API, and with a small retriever and a conversation history we get a working RAG assistant with a web interface. The measurements also showed the main trade-off of local AI: qwen3.5:9b gives the best answers, while gemma3:4b is twice as fast with half the memory and is still correct on simple tasks. The models also made mistakes (a wrong statement about Git, a wrong risk level in structured output), so the answers of a local model still have to be checked, and the server must stay on `localhost` or behind authentication.